Esteban Babadilla-Cruz Homework #2

Problem 1

Let $A[1 \ldots n]$ be an array of n distinct numbers. If $i<j$ and $A[i] > A[j]$, then the pair (i,j) is called a reverse pair for A

Part a) List the five reverse pairs of the array $A = [2,3,8,6,1]$

Assumptions on algorithm: there are two loops. Outer loop is index i and then the inner loop deals with index j since we are essentially dealing with all indices greater than i, which is every index following i until the very last index of the array. At every pair of indices (i,j) the algorithm will check whether $A[i] > A[j]$. If this statement returns true, we will consider this a reverse pair for A.

Thus the five reverse pairs of A are:

Index starts at 1

Indices (i,j) (where $i<j$)	Element values $A[i], A[j]$ (where $A[i] > A[j]$)
(1, 5)	(2, 1)
(2, 5)	(3, 1)
(3, 4)	(8, 6)
(3, 5)	(8, 1)
(4, 5)	(6, 1)

5 Reverse pairs (i, j)

Q1)

Part b) What is the relationship b/w the running Time of Insertion_Sort and the number of reverse Pairs in the input array?

The running time for average and worst case Scenarios for Insertion Sort is $\Theta(n^2)$. Since we are, in our algorithm undergoing two loops the time complexity for reverse pairs is on average and in worst case scenarios $\Theta(n^2)$. Thus allowing insertion sort to be represented by $f(n)$ and reverse pairs to be rep. by $g(n)$,

The relationship b/w their time complexities is

$$\left.\begin{array}{l} f = \Theta(g) \\ g = \Theta(f) \\ f(n) \simeq g(n) \end{array}\right\} \text{asymptotically equal}$$

Q1)

Part C) Modify Merge_sort in order to give an alg. which determines the num of reverse pairs in an array of n numbers in $\Theta(n \log n)$ run time.

Why it works

since we are looking for 2 conditions ($i<j$ and $A[i] > A[j]$) I just add a statement (☆) that checks if $i<j$ because we already know that $B[i] > C[j]$ thus this adds on a total of $O(1)$ to our running time which due to rules for computing growth rates is insignificant, leaving our tc to be $O(n \log n)$

psuedo code

```
Merge_sort(A[1...n])

if n ≤ 1
    terminate

merge_sort(A[1....n/2])
merge_sort(A[n/2+1...n])
return merge(A[1..n/2], A[n/2+1...n])

merge(A[1..n/2], A[n/2+1....n])
    define D[1...m+k] // m+k is total array size
        i,j = 1
        count = 0
    for l = 1 to m+k
        if j > k or (i ≤ m && B[i] ≤ C[j])
            D(l) = B[i]
            i = i+1
        else
            D[l] = C[j]   // B[j] was bigger so check indices
(☆)         if i < j:
                count = count + 1
            j = j+1
```

Q2

Part a) define $T(n)$ as the TC of this variant. Find a rec formula

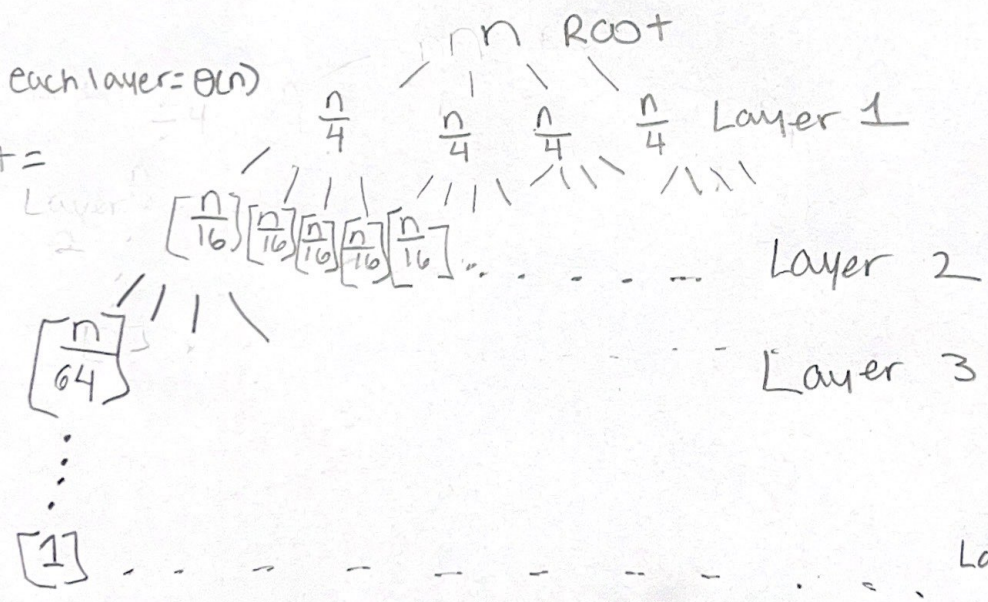
$$T(n) = O(1) + T(n/4) + T(n/4) + T(n/4) + T(n/4) + \Theta(n)$$

$$\boxed{T(n) = 4T(n/4) + \Theta(n)}$$

Part b) draw corresponding rec tree. Need height, layer rt, total layer sum

· TC of each layer $= \Theta(n)$

height =



Time complexity for each layer $\simeq n$

Last layer is $O(1)$ since array size $= 1$

but we have n $O(1)$ commands $\simeq n$

$$\#\text{Layers} = \frac{n}{(4 \times 4 \times \ldots \times 4)} \Rightarrow n = 4^{\#\text{layers}}$$

so height $= n = 4^x$

$\log_4 n = \log_4(4^x)$ used base 4 for simplicity

$\log_4 n = x$

TC = #iterations(layer) · TC of each iteration (layer)

thus $TC = n\,\Theta(\log_4 n)$ using log rules, $\simeq \Theta(n \log_4 n)$ for all cases (upper + Layer)

Q3

a) $T(n) = T(n-1) + n$

height $= O(n - \text{layer})$

n Root

$n-1$ Layer 1

$n-2$ Layer 2

$n-3$ Layer 3

$n-4$ Layer 4

$\vdots$

until no more elements

Thus $T(n) = O(n) + O(n-1) + O(n-2) + \ldots + 1$

$T(n) = O(n)$

---

b) $T(n) = \underset{\text{two times}}{2}\, T(\overbrace{n/4}^{\text{split array length by 4}}) + \sqrt{n}$

Assume $n = 4^k$

n Root

split into 2

$n/4$ $\quad$ $n/4$

$\div 2$

$\left[\frac{n}{16}\right]$ $\left[\frac{n}{16}\right]$ $\left[\frac{n}{16}\right]$ $\left[\frac{n}{16}\right]$

$\left[\frac{n}{32}\right]$ $\left[\frac{n}{32}\right]$ . . . . .

$\vdots$

$[1]$ . . . . . .

2 times $\quad$ TC of each

$2 \cdot \sqrt{\frac{n}{4}} \quad \sqrt{\frac{4n}{4}} = \sqrt{n}$

$4 \cdot \sqrt{\frac{n}{16}} = \sqrt{\frac{16n}{16}} = \sqrt{n}$

since $n = 4^k$

$k = \log_4 n$

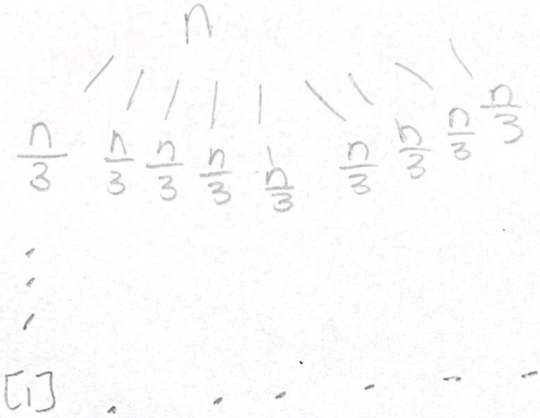
$T(n) = \#\text{ of layers} \cdot \sqrt{n} = (\sqrt{n})(\log_4 n)$

thus $T(n) = \Theta(\sqrt{n} \log_4 n)$

Q3) Part c

c) $T(n) = \overset{\text{a times}}{9}T(n/3) + n^2$

split by 3



Layer 1
n^2

even layer has TC n^2

$n = 9^x$

$\log_9 n = \log_9 x$

$x = \log_9 n$

Thus $TC = n^2 \log_9(n)$

$$\boxed{TC = \Theta(n^2 \log_9(n))}$$

Q3)

Part a)

$$\underline{T(n/2)} + 1$$

split into 2

n ROOT

|

n/2 Layer 1: O(1)

|

n/4 Layer 2: O(1)

⋮

[1]

height is constant just splitting in half

$n = 2^k$ thus $\log_2 n = k$

$T(n) = \log_2(n)$

$T(n) = \Theta(\log_2 n)$

Q4)

divide and conquer Alg to find largest number in A
Write in psuedo code and find running avg-

Quick Sort A[1...n]
// choose a pivot, A[n] for now
if $n \leq 1$
terminate
k = Partition (A[1...n], pivot = A[n])
Quick Sort (A[1...k-1])
Quicksort (A[k+1...n])

Partition function
define B[1...n]
// scan all elements, if it is larger we insert at end, if it is smaller than pivot we insert at beg.
// if it is bigger, we will mark it as max value
// when we find the value needed at A[n] that is our max num

Rt: O(logn)